

# **Отчёт по лабораторной работе №12**

**Программирование в командном процессоре UNIX**

Добрынин Никита Артёмович

# Содержание

|          |                                       |           |
|----------|---------------------------------------|-----------|
| <b>1</b> | <b>Цель работы</b>                    | <b>5</b>  |
| <b>2</b> | <b>Задание</b>                        | <b>6</b>  |
| <b>3</b> | <b>Выполнение лабораторной работы</b> | <b>7</b>  |
| <b>4</b> | <b>Ответы на вопросы</b>              | <b>11</b> |
| <b>5</b> | <b>Выводы</b>                         | <b>13</b> |

## Список иллюстраций

|     |                                                                     |    |
|-----|---------------------------------------------------------------------|----|
| 3.1 | Скрипт первой задачи в редакторе nano . . . . .                     | 7  |
| 3.2 | chmod +x first_quest . . . . .                                      | 7  |
| 3.3 | ./first_quest . . . . .                                             | 7  |
| 3.4 | Скрипт второй задачи в редакторе nano . . . . .                     | 8  |
| 3.5 | chmod +x second_quest, ./second_quest arg1 arg2 arg3 arg4 . . . . . | 8  |
| 3.6 | Скрипт третьей задачи в редакторе nano . . . . .                    | 9  |
| 3.7 | chmod +x third_quest, ./third_quest . . . . .                       | 9  |
| 3.8 | Скрипт четвертой задачи в редакторе nano . . . . .                  | 10 |
| 3.9 | ./fourth_quest . . . . .                                            | 10 |

## **Список таблиц**

# 1 Цель работы

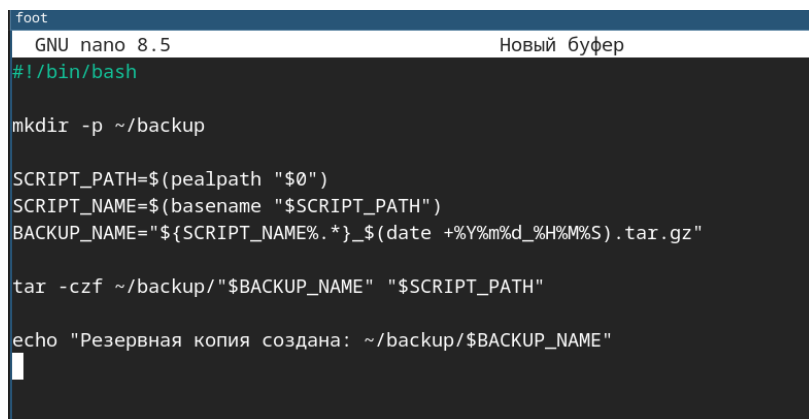
Изучить основы программирования в оболочке ОС UNIX/Linux. Научиться писать небольшие командные файлы.

## 2 Задание

1. Объясните понятие командной оболочки. Приведите примеры командных оболочек. Чем они отличаются?
2. Что такое POSIX?
3. Как определяются переменные и массивы в языке программирования bash?
4. Каково назначение операторов let и read?
5. Какие арифметические операции можно применять в языке программирования bash?
6. Что означает операция (( ))?
7. Какие стандартные имена переменных Вам известны?
8. Что такое метасимволы?
9. Как экранировать метасимволы?
10. Как создавать и запускать командные файлы?
11. Как определяются функции в языке программирования bash?
12. Каким образом можно выяснить, является файл каталогом или обычным файлом?
13. Каково назначение команд set, typeset и unset?
14. Как передаются параметры в командные файлы?
15. Назовите специальные переменные языка bash и их назначение.

### 3 Выполнение лабораторной работы

Написал скрипт который делает резервную копию самого себя(рис. 3.1).



```
foot
GNU nano 8.5                                Новый буфер
#!/bin/bash

mkdir -p ~/backup

SCRIPT_PATH=$(realpath "$0")
SCRIPT_NAME=$(basename "$SCRIPT_PATH")
BACKUP_NAME="${SCRIPT_NAME%.*}_${date +%Y%m%d_%H%M%S}.tar.gz"

tar -czf ~/backup/"$BACKUP_NAME" "$SCRIPT_PATH"

echo "Резервная копия создана: ~/backup/$BACKUP_NAME"
```

Рисунок 3.1: Скрипт первой задачи в редакторе nano

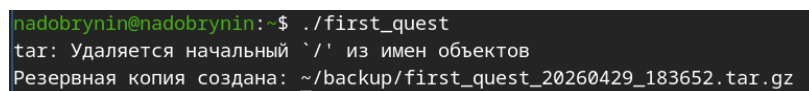
Сделал скрипт исполняемым(рис. 3.2).



```
nadobrynin@nadobrynin:~$ chmod +x first_quest
```

Рисунок 3.2: chmod +x first\_quest

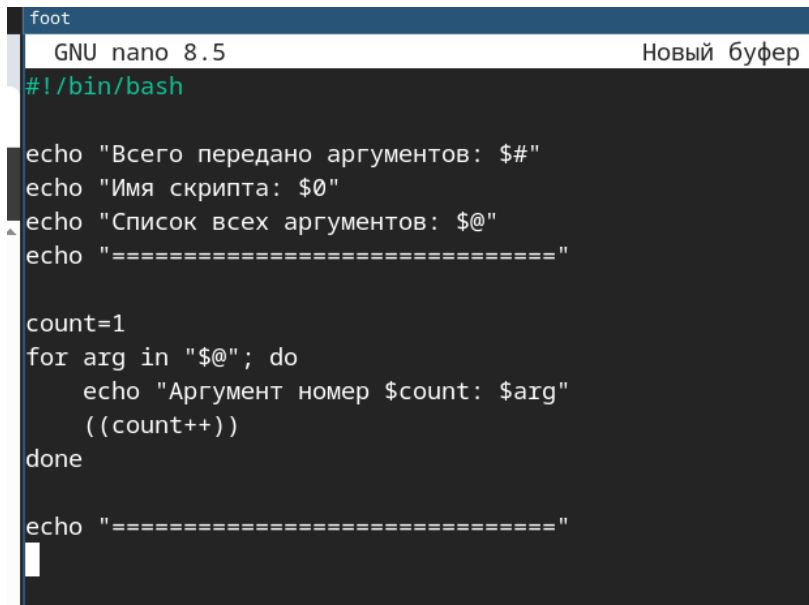
Запустил скрипт(рис. 3.3).



```
nadobrynin@nadobrynin:~$ ./first_quest
tar: Удаляется начальный '/' из имен объектов
Резервная копия создана: ~/backup/first_quest_20260429_183652.tar.gz
```

Рисунок 3.3: ./first\_quest

Написал скрипт который обрабатывает произвольное количество входящих аргументов(рис. 3.4).



```
foot
GNU nano 8.5                                Новый буфер
#!/bin/bash

echo "Всего передано аргументов: $#"
```

```
echo "Имя скрипта: $0"
```

```
echo "Список всех аргументов: $@"
```

```
echo "====="
```

```
count=1
```

```
for arg in "$@"; do
```

```
    echo "Аргумент номер $count: $arg"
```

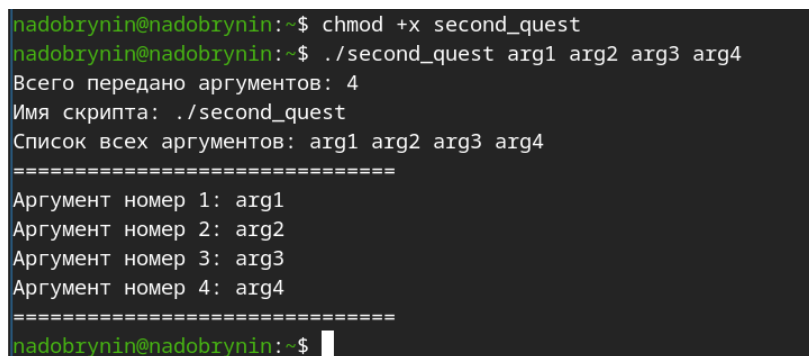
```
    ((count++))
```

```
done
```

```
echo "====="
```

Рисунок 3.4: Скрипт второй задачи в редакторе nano

Сделал файл исполняемым и запустил его(рис. 3.5).



```
nadobrynin@nadobrynin:~$ chmod +x second_quest
```

```
nadobrynin@nadobrynin:~$ ./second_quest arg1 arg2 arg3 arg4
```

```
Всего передано аргументов: 4
```

```
Имя скрипта: ./second_quest
```

```
Список всех аргументов: arg1 arg2 arg3 arg4
```

```
=====
```

```
Аргумент номер 1: arg1
```

```
Аргумент номер 2: arg2
```

```
Аргумент номер 3: arg3
```

```
Аргумент номер 4: arg4
```

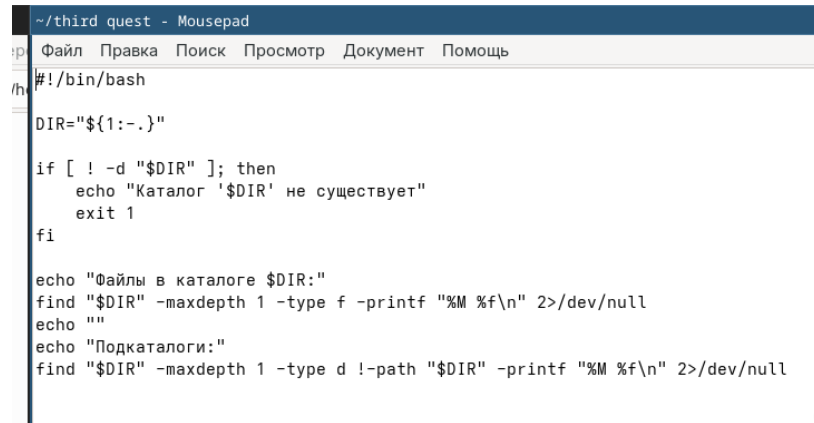
```
=====
```

```
nadobrynin@nadobrynin:~$
```

Рисунок 3.5: `chmod +x second_quest`, `./second_quest arg1 arg2 arg3 arg4`

Написал скрипт, аналог команды `ls`(рис. 3.6).



A screenshot of a nano editor window titled '~/.third\_quest - Mousepad'. The menu bar includes 'Файл', 'Правка', 'Поиск', 'Просмотр', 'Документ', and 'Помощь'. The script content is as follows:

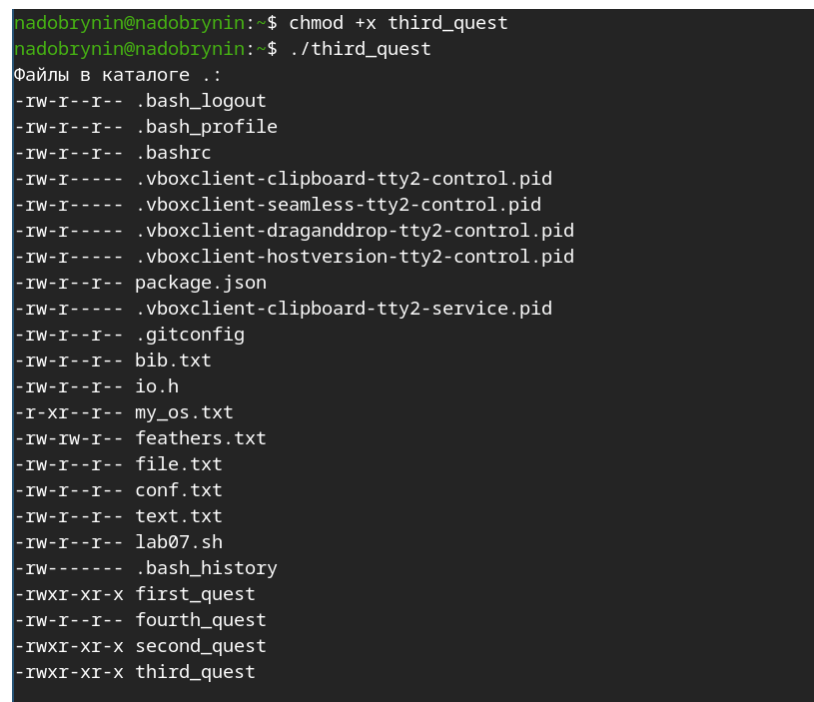
```
#!/bin/bash
DIR="${1:-.}"

if [ ! -d "$DIR" ]; then
    echo "Каталог '$DIR' не существует"
    exit 1
fi

echo "Файлы в каталоге $DIR:"
find "$DIR" -maxdepth 1 -type f -printf "%M %f\n" 2>/dev/null
echo ""
echo "Подкаталоги:"
find "$DIR" -maxdepth 1 -type d !-path "$DIR" -printf "%M %f\n" 2>/dev/null
```

Рисунок 3.6: Скрипт третьей задачи в редакторе nano

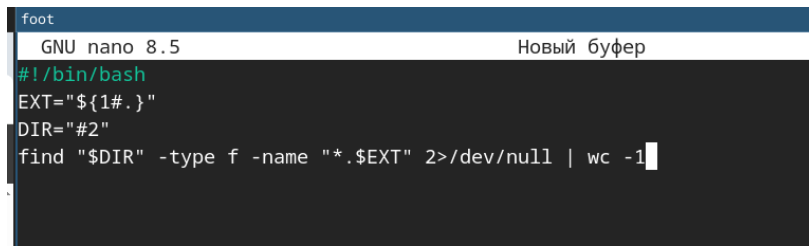
Сделал файл исполняемым и запустил его(рис. 3.7).

A screenshot of a terminal window showing the execution of the 'third\_quest' script. The user runs 'chmod +x third\_quest' and then './third\_quest'. The output lists files and subdirectories in the current directory with their permissions and names.

```
nadobrynin@nadobrynin:~$ chmod +x third_quest
nadobrynin@nadobrynin:~$ ./third_quest
Файлы в каталоге .:
-rw-r--r-- .bash_logout
-rw-r--r-- .bash_profile
-rw-r--r-- .bashrc
-rw-r----- .vboxclient-clipboard-tty2-control.pid
-rw-r----- .vboxclient-seamless-tty2-control.pid
-rw-r----- .vboxclient-draganddrop-tty2-control.pid
-rw-r----- .vboxclient-hostversion-tty2-control.pid
-rw-r--r-- package.json
-rw-r----- .vboxclient-clipboard-tty2-service.pid
-rw-r--r-- .gitconfig
-rw-r--r-- bib.txt
-rw-r--r-- io.h
-r-xr--r-- my_os.txt
-rw-rw-r-- feathers.txt
-rw-r--r-- file.txt
-rw-r--r-- conf.txt
-rw-r--r-- text.txt
-rw-r--r-- lab07.sh
-rw----- .bash_history
-rwxr-xr-x first_quest
-rw-r--r-- fourth_quest
-rwxr-xr-x second_quest
-rwxr-xr-x third_quest
```

Рисунок 3.7: chmod +x third\_quest, ./third\_quest

Написал простой скрипт который выводит количество указанных файлов в текущем каталоге(рис. 3.8).

A screenshot of the nano text editor interface. The title bar at the top shows 'foot' on the left and 'Новый буфер' (New buffer) on the right. Below the title bar, it says 'GNU nano 8.5'. The editor contains the following text: 

```
#!/bin/bash
EXT="${1#.}"
DIR="#2"
find "$DIR" -type f -name ".*$EXT" 2>/dev/null | wc -1
```

 The cursor is at the end of the last line.

Рисунок 3.8: Скрипт четвертой задачи в редакторе nano

Сделал его исполняемым и запустил(рис. 3.9).

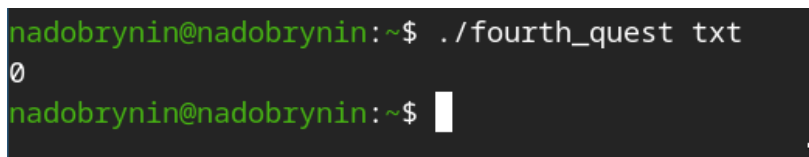
A screenshot of a terminal window. The prompt is 'nadobrynin@nadobrynin:~\$'. The user has entered './fourth\_quest txt' and pressed enter. The output is '0'. The prompt is now 'nadobrynin@nadobrynin:~\$' with a cursor at the end.

Рисунок 3.9: ./fourth\_quest

## 4 Ответы на вопросы

- 1) Командная оболочка (shell) - программа которая принимает команды пользователя и передает ее ОС Примеры: sh, bash, zsh, fish
- 2) POSIX - набор стандартов IEEE, определяющий единый интерфейс для UNIX-подобных ОС
- 3) Переменные определяются без знака \$
- 4) let - выполняет арифметические операции над целыми числами, read - считывает ввод пользователя и сохраняет в переменные
- 5) Сложение, вычитание, умножение, сравнения, логические операции и т. д.
- 6) Двойные круглые скобки используются для арифметических вычислений и проверок без вызова let
- 7) HOME, PATH, USER, SHELL, PWD, RANDOM, OLDPWD, UID.
- 8) Метасимволы - символы, имеющие спец значение для оболочки
- 9) Метасимвол можно экранировать как и обычный символ обратным слэшем - “\”
- 10) Создать файл с расширением .sh, добавить в файл «#!/» и путь к интерпретатору, сделать файл исполняемым

- 11) Командой `function`
- 12) Используя условные операторы `[]` или `[][]`
- 13) `set` - выводит параметры окружения и локальные переменные, `typeset` - используется для задания атрибутов переменной
- 14) При запуске параметры передаются через пробел после строки запуска скрипта
- 15) `$0` - имя скрипта, `$#` - кол-во параметров, `$@` - параметры как отдельные слова, `$*` - все параметры как одно слово, `$$` - PID текущего процесса оболочки

## **5 Выводы**

Я освоил базовые команды UNIX и написал 4 простых скрипта.